

DB Backlog

SI No	Feature	Story No	Description	Acceptance Criteria
1	User Table	DB0001	Create table to store registered user accounts and their profile details	Acceptance Criteria 1. Table Creation -A table named users should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -username VARCHAR(50) UNIQUE NOT NULL -password VARCHAR(255) NOT NULL -full_name VARCHAR(100) NOT NULL -email VARCHAR(100) UNIQUE NOT NULL -phone VARCHAR(15) -role VARCHAR(10) NOT NULL DEFAULT 'USER' -created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP 2. Data Validation -id should be auto-generated. -username must be unique and cannot be null. -email must be unique and match email format. -role must be one of: USER, ADMIN. -password must store BCrypt hash only. -Mandatory fields cannot be null.
2	User Role Enum	DB0002	Define allowable role values for user accounts to enforce access control	Acceptance Criteria 1. Enum Definition -Role column should only accept: USER, ADMIN. -Default value for new registrations must be USER. 2. Data Validation -Any insert with a role outside USER or ADMIN must be rejected with constraint violation. -First ADMIN must be bootstrapped via seed data or registration logic. -Role column cannot be null. -Role value persists correctly after update operations.
3	User Unique Constraints	DB0003	Add unique constraints on username and email to prevent duplicate registrations	Acceptance Criteria 1. Constraint Creation -UNIQUE constraint on users.username. -UNIQUE constraint on users.email. 2. Data Validation -Inserting a duplicate username returns a unique constraint violation. -Inserting a duplicate email returns a unique constraint violation. -Updating to an existing username or email is also rejected. -Constraints survive table truncation and re-seeding.
4	User Indexes	DB0004	Create indexes on frequently queried user fields to improve lookup performance	Acceptance Criteria 1. Index Creation -Index on users.username for login queries. -Index on users.email for email lookup. -Index on users.role for admin filtering. 2. Validation -Query EXPLAIN on login by username shows index usage. -Query EXPLAIN on filter by role shows index usage. -Index creation does not block existing data reads. -Index names follow convention: idx_users_.

SI No	Feature	Story No	Description	Acceptance Criteria
5	Car Table	DB0005	Create table to store all car listings submitted by sellers	Acceptance Criteria 1. Table Creation -A table named cars should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -make VARCHAR(50) NOT NULL -model VARCHAR(50) NOT NULL -year INT NOT NULL -price DECIMAL(12,2) NOT NULL -color VARCHAR(30) -fuel_type VARCHAR(20) -transmission VARCHAR(20) -mileage INT -description TEXT -available BOOLEAN DEFAULT TRUE -approval_status VARCHAR(20) DEFAULT 'PENDING' -seller_id BIGINT (FK → users.id) -created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP 2. Data Validation -id must be auto-generated. -year must be between 1900 and current year + 1. -price must be greater than 0. -available defaults to TRUE on creation. -approval_status defaults to PENDING.
6	Car ApprovalStatus Enum	DB0006	Define approval lifecycle values for car listings requiring admin moderation	Acceptance Criteria 1. Enum Definition -approval_status column accepts only: PENDING, APPROVED, REJECTED. -Default value on insert must be PENDING. 2. Data Validation -Listing cannot be purchased unless approval_status is APPROVED. -Rejected listing cannot be re-approved without new submission. -Status transitions must follow: PENDING → APPROVED or PENDING → REJECTED only. -Any value outside the three allowed values is rejected.
7	Car Availability Flag	DB0007	Track whether a listed car is still available for purchase after a sale is completed	Acceptance Criteria 1. Column Definition -available BOOLEAN NOT NULL DEFAULT TRUE on cars table. 2. Data Validation -available is set to FALSE atomically when a purchase order is confirmed. -Duplicate purchase of unavailable car returns 409 Conflict. -available cannot be manually set to TRUE by seller after a completed sale. -available flag persists correctly across server restarts. -Rollback of a failed purchase resets available to TRUE.
8	Car Indexes	DB0008	Create indexes on cars table for browse	filter

SI No	Feature	Story No	Description	Acceptance Criteria
9	Car Seller Foreign Key	DB0009	Enforce referential integrity between cars and the users who listed them	Acceptance Criteria 1. Foreign Key Definition -cars.seller_id references users.id. -ON DELETE behavior: SET NULL or RESTRICT (no orphaned listings). 2. Data Validation -Inserting a car with a non-existent seller_id returns FK violation. -Deleting a user who has active listings is restricted. -seller_id is indexed to support FK lookups. -Cascading behaviour documented and tested.
10	Session Token Table	DB0010	Create table to store active user session tokens for stateless authentication	Acceptance Criteria 1. Table Creation -A table named session_tokens should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -token VARCHAR(255) UNIQUE NOT NULL -user_id BIGINT NOT NULL (FK → users.id) -created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP -expires_at TIMESTAMP NOT NULL -is_active BOOLEAN DEFAULT TRUE 2. Data Validation -token must be unique and non-null. -expires_at must be calculated as created_at + 120 minutes. -Expired tokens must return 401 on validation. -Logout sets is_active to FALSE immediately. -Token cannot be reused after logout.
11	Session Expiry Logic	DB0011	Enforce automatic expiration of session tokens after 120 minutes of inactivity	Acceptance Criteria 1. Expiry Behaviour -expires_at set to created_at + SESSION_EXPIRATION_MINUTES (default 120). -Each valid request extends expires_at by SESSION_EXPIRATION_MINUTES. -Request with expires_at in the past returns 401 Unauthorized. 2. Cleanup Job -Scheduled task deletes all rows where is_active = FALSE or expires_at < NOW() every 30 minutes. -Cleanup does not affect active sessions. -Cleanup runs on application startup if stale tokens exist.
12	Session Index	DB0012	Index the session token column for fast authentication lookup on every request	Acceptance Criteria 1. Index Creation -UNIQUE index on session_tokens.token. -Index on session_tokens.user_id for user session lookup. -Index on session_tokens.expires_at for cleanup job efficiency. 2. Validation -Auth lookup by token completes in under 5 ms on 10,000 row dataset. -Index present in SHOW INDEXES output after schema init. -Index names: idx_session_token, idx_session_user_id, idx_session_expires.

SI No	Feature	Story No	Description	Acceptance Criteria
13	Purchase Order Table	DB0013	Create table to record all car purchase transactions between buyers and sellers	Acceptance Criteria 1. Table Creation -A table named purchase_orders should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -car_id BIGINT NOT NULL (FK → cars.id) -buyer_id BIGINT NOT NULL (FK → users.id) -seller_id BIGINT NOT NULL (FK → users.id) -order_status VARCHAR(30) DEFAULT 'PENDING_ADMIN_APPROVAL' -payment_status VARCHAR(20) DEFAULT 'PENDING' -total_amount DECIMAL(12,2) NOT NULL -fraud_alert BOOLEAN DEFAULT FALSE -created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP -updated_at TIMESTAMP 2. Data Validation -buyer_id and seller_id cannot be the same user. -car_id must reference an existing available car. -total_amount must match cars.price at time of order. -order_status defaults to PENDING_ADMIN_APPROVAL.
14	Order Status Enum	DB0014	Define the multi-step approval lifecycle for purchase orders through admin and seller stages	Acceptance Criteria 1. Enum Values -order_status accepts: PENDING_ADMIN_APPROVAL, ADMIN_APPROVED, ADMIN_REJECTED, PENDING_SELLER_APPROVAL, SELLER_APPROVED, SELLER_REJECTED, COMPLETED, CANCELLED. 2. Transition Rules -New order must start at PENDING_ADMIN_APPROVAL. -Admin approval moves to PENDING_SELLER_APPROVAL. -Admin rejection moves to ADMIN_REJECTED (terminal). -Seller approval moves to COMPLETED. -Seller rejection moves to SELLER_REJECTED (terminal). -No backward transitions allowed. -car.available set to FALSE only when COMPLETED.
15	Payment Status Enum	DB0015	Track payment state for each purchase order from initiation through confirmation	Acceptance Criteria 1. Enum Values -payment_status accepts: PENDING, PAID, FAILED, REFUNDED. 2. Data Validation -Default value on order creation is PENDING. -Successful payment gateway response sets status to PAID. -Failed gateway response sets status to FAILED and rolls back car availability. -Refunded status is set only on SELLER_REJECTED or ADMIN_REJECTED orders. -payment_status cannot be changed once set to REFUNDED.

SI No	Feature	Story No	Description	Acceptance Criteria
16	Payment Details Table	DB0016	Store payment method and transaction token details for each purchase order	Acceptance Criteria 1. Table Creation -A table named payment_details should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -order_id BIGINT UNIQUE NOT NULL (FK → purchase_orders.id) -payment_method VARCHAR(20) NOT NULL -payment_token VARCHAR(255) NOT NULL -gateway_response VARCHAR(255) -paid_at TIMESTAMP 2. Data Validation -payment_method accepts: CREDIT_CARD, UPI, NET_BANKING only. -payment_token must not be null. -One payment record per order (order_id UNIQUE). -paid_at populated only when payment_status = PAID. -Records are immutable after creation — no UPDATE allowed.
17	Order Indexes	DB0017	Index purchase_orders for buyer	seller
18	Order Foreign Keys	DB0018	Enforce referential integrity between orders and their related cars and users	Acceptance Criteria 1. FK Definitions -purchase_orders.car_id → cars.id ON DELETE RESTRICT. -purchase_orders.buyer_id → users.id ON DELETE RESTRICT. -purchase_orders.seller_id → users.id ON DELETE RESTRICT. 2. Validation -Creating an order with a non-existent car_id is rejected. -Creating an order with a non-existent buyer_id is rejected. -Deleting a user with linked orders is restricted. -Deleting a car with linked orders is restricted. -All FKs verified on schema load.
19	Car Availability Atomicity	DB0019	Ensure car availability is updated atomically within the same transaction as order creation	Acceptance Criteria 1. Transaction Behaviour -cars.available is set to FALSE in the same DB transaction as purchase_orders INSERT. -If order INSERT fails, cars.available remains TRUE (full rollback). -If availability update fails, entire transaction rolls back. 2. Concurrency -Concurrent purchases of the same car_id: only one succeeds; second receives 409. -Tested with 10 simultaneous purchase requests. -No dirty reads between concurrent transactions. -Database-level lock prevents double-booking.

SI No	Feature	Story No	Description	Acceptance Criteria
20	Wishlist Table	DB0020	Create table to allow users to save cars they are interested in purchasing later	Acceptance Criteria 1. Table Creation -A table named wishlist_items should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -user_id BIGINT NOT NULL (FK → users.id) -car_id BIGINT NOT NULL (FK → cars.id) -added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP 2. Data Validation -Composite UNIQUE constraint on (user_id, car_id) — no duplicate entries. -Adding same car twice returns 409 or is idempotent. -Deleting a car cascades to remove its wishlist entries. -Deleting a user cascades to remove their wishlist. -Wishlist persists across browser sessions.
21	Wishlist Unique Constraint	DB0021	Prevent duplicate wishlist entries for the same user-car combination	Acceptance Criteria 1. Constraint Definition -UNIQUE KEY uq_wishlist_user_car (user_id, car_id) on wishlist_items. 2. Validation -Second insert of same (user_id, car_id) raises duplicate key error. -Frontend handles this with a 409 response without crashing. -Removing and re-adding the same car is allowed after removal. -Constraint verified in SHOW CREATE TABLE output.
22	Wishlist Cascade Delete	DB0022	Ensure wishlist entries are cleaned up when a car or user is deleted from the system	Acceptance Criteria 1. FK Cascade Rules -wishlist_items.user_id → users.id ON DELETE CASCADE. -wishlist_items.car_id → cars.id ON DELETE CASCADE. 2. Validation -Deleting a user removes all their wishlist rows automatically. -Deleting a car removes all wishlist rows referencing it. -No orphaned wishlist rows remain after cascade. -Cascade does not affect other users' wishlists when one user is deleted.

SI No	Feature	Story No	Description	Acceptance Criteria
23	Feedback Table	DB0023	Create table to store user-submitted feedback about the platform experience	Acceptance Criteria 1. Table Creation -A table named feedback should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -user_id BIGINT (FK → users.id, nullable for anonymous) -rating INT NOT NULL -message TEXT NOT NULL -submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP 2. Data Validation -rating must be between 1 and 5 inclusive. -message cannot be null or empty. -user_id is nullable to allow anonymous submissions. -Admin can view all feedback via GET /api/feedback/admin. -Feedback is read-only after submission (no UPDATE).
24	Feedback Rating Constraint	DB0024	Add check constraint to ensure feedback ratings fall within the valid 1–5 range	Acceptance Criteria 1. Constraint Definition -CHECK (rating >= 1 AND rating <= 5) on feedback.rating. 2. Validation -Inserting rating = 0 raises constraint violation. -Inserting rating = 6 raises constraint violation. -Ratings 1 through 5 are all accepted. -Constraint is visible in SHOW CREATE TABLE output. -Constraint enforced at DB layer independent of application validation.
25	Support Ticket Table	DB0025	Create table to manage customer support tickets raised by users for issue resolution	Acceptance Criteria 1. Table Creation -A table named support_tickets should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -user_id BIGINT NOT NULL (FK → users.id) -subject VARCHAR(255) NOT NULL -ticket_status VARCHAR(20) DEFAULT 'OPEN' -created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP -updated_at TIMESTAMP 2. Data Validation -subject cannot be null or empty. -ticket_status defaults to OPEN on creation. -Only the ticket owner or admin may view the ticket. -updated_at refreshes on every status change or new message.

SI No	Feature	Story No	Description	Acceptance Criteria
26	Ticket Status Enum	DB0026	Define ticket lifecycle status values from open through resolution to closed	Acceptance Criteria 1. Enum Values -ticket_status accepts: OPEN, IN_PROGRESS, RESOLVED, CLOSED. 2. Transition Rules -All new tickets start as OPEN. -Admin reply moves status to IN_PROGRESS. -Admin can move ticket to RESOLVED or CLOSED. -User cannot directly close their own ticket. -CLOSED is a terminal state — no further messages allowed. -Any value outside the four allowed values is rejected by DB constraint.
27	Ticket Message Table	DB0027	Create table to store threaded messages within a support ticket for bidirectional communication	Acceptance Criteria 1. Table Creation -A table named ticket_messages should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -ticket_id BIGINT NOT NULL (FK → support_tickets.id) -sender_id BIGINT NOT NULL (FK → users.id) -message TEXT NOT NULL -sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP 2. Data Validation -message cannot be null or empty. -Only ticket owner or ADMIN role can insert messages. -Messages returned in ascending sent_at order. -Deleting a ticket cascades to delete all its messages. -No message edits or deletes allowed after creation.
28	Ticket Indexes	DB0028	Index support ticket and message tables for user and admin query performance	Acceptance Criteria 1. Index Creation -Index on support_tickets.user_id for GET /api/user/support-tickets. -Index on support_tickets.ticket_status for admin open ticket dashboard. -Index on ticket_messages.ticket_id for message thread retrieval. -Index on ticket_messages.sent_at for chronological ordering. 2. Validation -Thread query for 100 messages returns in under 10 ms. -EXPLAIN shows index usage on ticket_id filter. -Index names: idx_tickets_user, idx_tickets_status, idx_tmsg_ticket.

SI No	Feature	Story No	Description	Acceptance Criteria
29	Seller Review Table	DB0029	Create table to store buyer ratings and reviews for sellers after completed transactions	Acceptance Criteria 1. Table Creation -A table named seller_reviews should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -seller_id BIGINT NOT NULL (FK → users.id) -reviewer_id BIGINT NOT NULL (FK → users.id) -order_id BIGINT NOT NULL (FK → purchase_orders.id) -rating INT NOT NULL -review_text TEXT -created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP 2. Data Validation -reviewer_id cannot equal seller_id (no self-review). -rating must be between 1 and 5. -UNIQUE constraint on (reviewer_id, seller_id) — one review per pair. -Reviewer must have a COMPLETED order with the seller (enforced at service layer). -Review cannot be submitted for a non-completed order.
30	Review Rating Constraint	DB0030	Enforce database-level constraint that seller review ratings stay within 1 to 5	Acceptance Criteria 1. Constraint Definition -CHECK (rating >= 1 AND rating <= 5) on seller_reviews.rating. -UNIQUE KEY uq_review_buyer_seller (reviewer_id, seller_id). 2. Validation -Duplicate review by same buyer for same seller updates existing record (upsert). -Rating 0 and rating 6 both raise constraint violations. -Average seller rating computed correctly across multiple reviews. -Constraint survives data migration and re-import.
31	Recent View Table	DB0031	Track the last 10 cars viewed by each user for the recent views feature on the browse page	Acceptance Criteria 1. Table Creation -A table named recent_views should be created. Fields: -id BIGINT AUTO_INCREMENT (Primary Key) -user_id BIGINT NOT NULL (FK → users.id) -car_id BIGINT NOT NULL (FK → cars.id) -viewed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP 2. Data Validation -New view for already-viewed car updates viewed_at rather than inserting duplicate. -Only the 10 most recent rows per user are retained; older rows auto-deleted. -Admins are excluded from view tracking. -Cascade delete when user or car is deleted.

SI No	Feature	Story No	Description	Acceptance Criteria
32	Recent View Limit Logic	DB0032	Implement a mechanism to retain only the 10 most recent views per user in the recent_views table	Acceptance Criteria 1. Cleanup Logic -On each insert, check count of recent_views for user_id. -If count exceeds 10, delete the oldest row(s) by viewed_at. -Logic implemented as application-level service or DB trigger. 2. Validation -After 15 sequential views by one user, only 10 rows exist for that user. -Rows deleted are always the oldest (lowest viewed_at). -Cleanup does not affect other users' recent view rows. -Performance: cleanup runs in same transaction as insert.
33	Recent View Indexes	DB0033	Index the recent_views table for fast user-specific retrieval and cleanup queries	Acceptance Criteria 1. Index Creation -Index on recent_views.user_id. -Index on recent_views.viewed_at for cleanup ORDER BY query. -Composite index on (user_id, viewed_at) for combined filter-sort. 2. Validation -GET /api/cars/recent query returns in under 10 ms for 1000 user rows. -Cleanup query uses index (verified via EXPLAIN). -Index: idx_rviews_user_time on (user_id, viewed_at).
34	Fraud Alert Flag	DB0034	Add a fraud alert flag to purchase orders to mark buyers with high purchase volume for admin review	Acceptance Criteria 1. Column Definition -fraud_alert BOOLEAN DEFAULT FALSE on purchase_orders. 2. Business Rule -fraud_alert is set to TRUE if buyer has more than 3 COMPLETED orders at time of new order creation. -Only COMPLETED orders count toward the threshold. -Threshold is configurable via application property. -Admin dashboard displays count of fraud-flagged orders. -fraud_alert flag persists and is not auto-cleared.
35	Fraud Alert Index	DB0035	Index the fraud_alert column to support efficient admin dashboard fraud monitoring queries	Acceptance Criteria 1. Index Creation -Index on purchase_orders.fraud_alert. 2. Validation -Admin query for flagged orders returns in under 20 ms on 10,000 order dataset. -EXPLAIN on WHERE fraud_alert = TRUE shows index usage. -Index does not slow down normal order creation inserts. -Index name: idx_orders_fraud.
36	Admin Audit Log Table	DB0036	Create table to record all admin actions performed on cars	orders

SI No	Feature	Story No	Description	Acceptance Criteria
37	Dashboard Aggregation View	DB0037	Create a database view or query to power the admin dashboard summary statistics efficiently	Acceptance Criteria 1. View / Query Definition -total_cars: COUNT of all cars. -available_cars: COUNT where available = TRUE. -pending_cars: COUNT where approval_status = PENDING. -total_users: COUNT of all users with role = USER. -total_orders: COUNT of all purchase_orders. -completed_orders: COUNT where order_status = COMPLETED. -total_revenue: SUM of total_amount where order_status = COMPLETED. -pending_orders: COUNT where order_status = PENDING_ADMIN_APPROVAL. 2. Validation -Dashboard query executes in under 100 ms on production-sized dataset. -All counts match direct table queries. -View refreshes dynamically on each request.
38	Orders Per Day Query	DB0038	Create a time-series aggregation query for the admin analytics chart showing daily order counts over the last 30 days	Acceptance Criteria 1. Query Definition -Group purchase_orders by DATE(created_at) for the past 30 days. -Output columns: date DATE, order_count INT. -Include days with 0 orders (fill gaps with 0). -Sort by date ASC. 2. Validation -Returns exactly 30 rows even with missing days. -Date labels in ISO 8601 format (YYYY-MM-DD). -Query executes in under 200 ms on 10,000 order dataset. -Used by /api/admin/dashboard endpoint.
39	Revenue Per Day Query	DB0039	Create a daily revenue aggregation query for the admin analytics chart over the last 30 days	Acceptance Criteria 1. Query Definition -Group COMPLETED purchase_orders by DATE(created_at). -Output columns: date DATE, daily_revenue DECIMAL. -Include days with 0 revenue (fill gaps with 0.00). -Sort by date ASC. 2. Validation -Returns exactly 30 rows for the trailing 30 days. -Revenue values match manual SUM checks. -COMPLETED filter correctly excludes rejected and pending orders. -Query result drives revenue chart in admin dashboard.

SI No	Feature	Story No	Description	Acceptance Criteria
40	Car Listings Per Day Query	DB0040	Create a daily car listing count query for the admin analytics chart showing listing trends	Acceptance Criteria 1. Query Definition -Group cars by DATE(created_at) for the past 30 days. -Output columns: date DATE, listing_count INT. -Include days with zero listings (fill gaps with 0). -Sort by date ASC. 2. Validation -Returns exactly 30 rows. -Listing counts match COUNT(*) queries for individual dates. -Query executes in under 100 ms. -Powers car listings trend chart in admin dashboard.
41	Car Condition Fields	DB0041	Add additional condition and specification fields to the cars table to support the car comparison feature	Acceptance Criteria 1. Column Additions to cars table -condition VARCHAR(20) (NEW, USED, CERTIFIED_PRE_OWNED) -body_type VARCHAR(30) -engine_cc INT -number_of_owners INT DEFAULT 1 -insured BOOLEAN DEFAULT FALSE -puc_valid BOOLEAN DEFAULT FALSE 2. Data Validation -number_of_owners must be >= 1. -engine_cc must be > 0 if provided. -condition must be one of the three allowed values. -All new columns nullable for backward compatibility. -Comparison endpoint returns all condition fields populated.
42	Car Compare Query	DB0042	Optimise the car comparison query to return two complete car records including all condition fields in one round trip	Acceptance Criteria 1. Query Definition -SELECT all car fields for two car IDs in a single query. -JOIN with users table to include seller username and rating. -No lazy-loaded fields — all condition columns eagerly fetched. 2. Validation -Response includes: make, model, year, price, color, fuel_type, transmission, body_type, engine_cc, number_of_owners, insured, puc_valid, condition. -Query returns in under 50 ms. -Returns 404 if either car_id does not exist. -Seller info included in response.
43	DB Seeding — Admin User	DB0043	Seed the database with a default admin account to bootstrap the system on first run	Acceptance Criteria 1. Seed Data -Insert one user with role = ADMIN. -username: demoadmin -password: BCrypt hash of Demo@1234x -email: admin@carbonmarket.com -full_name: Demo Admin 2. Validation -Seed runs only if no ADMIN user exists (idempotent). -Admin can log in with demoadmin / Demo@1234x after seed. -Seed executed via ApplicationRunner or CommandLineRunner on startup. -Seed does not re-run on subsequent restarts if admin already present.

SI No	Feature	Story No	Description	Acceptance Criteria
44	DB Seeding — Demo User	DB0044	Seed the database with a demo buyer/seller user account for testing and demonstration purposes	Acceptance Criteria 1. Seed Data -Insert one user with role = USER. -username: demouser -password: BCrypt hash of Demo@1234 -email: user@carbonmarket.com -full_name: Demo User 2. Validation -Seed runs only if demouser does not already exist (idempotent). -User can log in with demouser / Demo@1234 after seed. -User has role USER and cannot access admin endpoints. -Seed coexists with admin seed in same startup runner.
45	DB Seeding — Demo Car Listings	DB0045	Seed the database with a set of sample car listings to populate the browse page for demonstration	Acceptance Criteria 1. Seed Data -Insert minimum 10 cars linked to demouser as seller. -Each car has: make, model, year, price, color, fuel_type, transmission, mileage, approval_status = APPROVED, available = TRUE. -Variety: at least 3 different makes, 2 fuel types, 2 transmission types. 2. Validation -Seeded cars appear in GET /api/cars/available response. -Seed is idempotent — does not insert duplicates on restart. -All 10 cars have approval_status = APPROVED. -Seeded cars have realistic price values (> 0).
46	Cascade Delete Rules	DB0046	Document and enforce cascade delete rules across all tables to maintain referential integrity on user deletion	Acceptance Criteria 1. Cascade Rules -Deleting a user cascades to: session_tokens, wishlist_items, recent_views, feedback (nullable → SET NULL), ticket_messages (sender_id → RESTRICT). -Deleting a car cascades to: wishlist_items, recent_views. -purchase_orders, support_tickets, seller_reviews: RESTRICT deletion of referenced users and cars. 2. Validation -Deleting a test user removes all their sessions, wishlist, and recent views. -Attempting to delete a user with open orders returns FK violation. -No orphaned rows remain in any child table after cascades. -All cascade rules documented in schema migration file.

SI No	Feature	Story No	Description	Acceptance Criteria
47	Database Connection Pool Config	DB0047	Configure HikariCP connection pool settings for optimal performance under concurrent load	Acceptance Criteria 1. Configuration -maximum-pool-size: 10 (configurable via env var). -minimum-idle: 2. -connection-timeout: 30000 ms. -idle-timeout: 600000 ms. -max-lifetime: 1800000 ms. 2. Validation -Application starts without connection timeout errors. -Concurrent 50-user test completes without pool exhaustion. -Pool metrics visible in actuator/health endpoint. -Config overridable via DB_POOL_SIZE environment variable.
48	Derby to MySQL Migration Config	DB0048	Ensure schema and all queries are compatible with both Apache Derby and MySQL via environment variable switching	Acceptance Criteria 1. Configuration -Default datasource: jdbc:derby:usedcarsdb;create=true (no credentials). -MySQL override via: DB_URL, DB_USERNAME, DB_PASSWORD, DB_DRIVER env vars. -JPA_DDL_AUTO env var controls schema creation (default: update). 2. Validation -Application starts with Derby config without any env vars set. -Application connects to MySQL when all four DB_* vars are set. -All JPA queries use standard JPQL — no Derby-specific or MySQL-specific SQL. -Schema auto-creates on first run with both databases. -Data types compatible: VARCHAR, INT, BOOLEAN, DECIMAL, TIMESTAMP.
49	JPA DDL Auto Config	DB0049	Control schema generation behaviour through the JPA_DDL_AUTO environment variable for different deployment environments	Acceptance Criteria 1. Configuration -JPA_DDL_AUTO defaults to update in application.properties. -Overridable at runtime via JPA_DDL_AUTO env var. -Accepted values: create, create-drop, update, validate, none. 2. Validation -update adds new columns without dropping existing data. -create-drop used in test environment (H2 in-memory). -validate fails fast if schema does not match entities. -none used in production to prevent accidental schema changes. -Test profile always uses H2 with create-drop.

SI No	Feature	Story No	Description	Acceptance Criteria
50	H2 In-Memory Test Database	DB0050	Configure H2 in-memory database for the test profile to ensure tests run in isolation without affecting production data	Acceptance Criteria 1. Test Configuration - spring.datasource.url=jdbc:h2:mem:testdb in test/resources/application.properties. -spring.jpa.hibernate.ddl-auto=create-drop for tests. -H2 console disabled in test profile. -All Spring Data JPA repositories work identically on H2 and Derby. 2. Validation -mvn test completes without connecting to Derby or MySQL. -Each test class starts with a clean schema (create-drop). -No test data leaks between test classes. -H2 dependency scoped as test-only in pom.xml. -All 50 DB backlog features testable via integration tests on H2.